Author(s): dywoq <dywoq.contact@gmail.com>

Revisions:
Aug-22 2026 – Creation date

1. Overview

This specification describes the IR subsystem's purpose, goals, procedure and API.

The IR (**I**nitialization **R**outine) is a kernel subsystem responsible for initializing the kernel. It relies on initialization functions of the kernel subsystems.

2. Goals

- Provide a centralized entry point that initializes and starts the kernel.
- Provide a mechanism to inspect the public information of the IR kernel subsystem.

3. Information

Internally, the IR subsystem contains parameters and additional meta information of the kernel subsystems, needed by the initialization procedure. The following table explicitly defines permissions of the IR subsystem and kernel subsystems, and describes the parameters' purpose and scope.

Name	Visibility Scope	Purpose
Condition	Public	This parameter provides a state of the initialization procedure.

- **FAILURE** → Some of the kernel subsystems failed to initialize. **Restrictions:** The kernel cannot be started. The SUCCESS, STARTED and READY conditions cannot be activated.
- **SUCCESS** → The kernel environment has been installed and subsystems have initialized successfully, and the kernel can be started. **Restrictions:** The initialization procedure cannot be started. The FAILURE and READY conditions cannot be activated.
- **STARTED** → The kernel have been started. **Restrictions:** The initialization procedure cannot be started. The SUCCESS, FAILURE and READY conditions cannot be activated.
- **READY** → The subsystem has not initialized yet and can start the initialization procedure. This is the initial condition. **Restrictions:** The STARTED condition cannot be activated.

The kernel subsystems are allowed to read it through using the public API, but not to modify it. The IR subsystem is allowed to read and

		modify it, but forbidden from activating more than one condition. Specific condition from the list cannot be forced despite the restrictions.
Initialization priority of the kernel subsystems	Private	This parameter is a list containing initialization priority of the kernel subsystems. The initialization priority parameter prevents circular dependency issues, since the kernel subsystems use others' functionality. The following rules are used to determine the initialization priority of a kernel subsystem: • As the kernel subsystem's depedencies list increases, its initialization priority decreases. • Kernel subsystem's dependencies are initialized first. Dependencies of the kernel subsystems are explicitly defined in their specifications.

3.1. Table - The IR Subsystem parameters

4. Initialization Procedure

The initialization procedure installs the kernel environment and initializes the kernel subsystems.

It consists of the following steps: **Environment installation, Subsystems Initialization** and **Startup**. They are done sequentially. All steps are to explicitly define their purpose, error cases and corresponding next step.

4.1. Environment installation

Purpose: This step installs the environment required by the kernel subsystems to function properly. The kernel environment implies that the segment registers (such as DS, SS, ES, CS) and stack pointer are provided with required values, which are shown below.

Object	**Value**	**Notes**
Code Segment Register	0x1000	This segment register is filled by the boot loader, when it performs a far jump to the kernel.
Data Segment Register	0x1600	*None*
Stack Segment Register	0x2000	*None*
Extra Segment Register	<See the notes>	This segment register replicates the DS parameters. Therefore, the kernel strictly cannot modify memory that

		belongs to the DS, using the ES.
Stack Pointer	0x0000	*None*

Error cases:

- Repeated installation of the kernel environment leads to undefined behavior.

Next step: **Subsystems Initialization**

4.2. Subsystems Initialization

Purpose: This step initializes the kernel subsystems. It relies on the "**Initialization priority of the kernel subsystems**" list to initialize the kernel subsystems in correct order (see the "3. Information" chapter).

If the kernel subsystems have initialized successfully, the subsystem activates the SUCCESS condition.

Error cases:

- If one of the kernel subsystems failed to initialize, the subsystem activates the "FAILURE" condition. The step does interrupts the initialization.
- If the kernel subsystems have been initialized and the current condition is SUCCESS, or they have failed to initialize and the current condition is FAILURE, the subsystem reports it and halts machine.

Next step: **Startup**

4.3. Startup

Purpose: This step starts the kernel loop and activates the "STARTED" condition.

Error cases:

- If the condition FAILURE is active, the subsystem prints a corresponding failure message and halts a machine.
- If the kernel has been already started and the current condition is STARTED, the subsystem reports it and halts machine.

Next step: *None*

5. API

5.1. Table

Name	Visibility Scope
IrEnvironmentInstallation	Private
IrSubsystemsInitialization	Private
IrStartup	Private
IrInspectInformation	Public

5.1.1. Table – API Functions

5.2. Environment installation

Visibility Scope: Private
Signature:

VOID
IrEnvironmentInstallation();

This function corresponds to the Initialization Procedure "**Environment Installation**" step. It is implemented fully in Assembly.

The function invokes the **IrSubsystemsInitialization** function after installing the kernel environment. The

IrEnvironmentInstallation function is to be placed at the very start of a compiled kernel.

The function must be not called by any kernel function and implemented fully in Assembly to prevent stack usage. Before the function provides the stack pointer and segment with the required values (see the "4.1. Environment Installation" chapter), they still contain values stored by the boot loader primary stage.

Repeated calling the function leads to undefined behavior.

5.3. Subsystems Initialization

Visibility Scope: Private
Signature:

VOID
IrSubsystemsInitialization();

This function corresponds to the Initialization Procedure "**Subsystems Initialization**" step.

If the function **IrSubsystemsInitialization** has been called twice, it may lead to halting a machine if the current condition is FAILURE and SUCCESS. The failure message to print depends on the condition:

- FAILURE → "Vacui: Kernel components have already failed to initialize. Halting."
- SUCCESS → "Vacui: Kernel components have already initialized successfully. Halting."

5.4. Startup

Visibility Scope: Private
Signature:

```
VOID
IrStartup();
```

This function corresponds to the Initialization Procedure "**Startup**" step. It is implemented using C and some inline assembly code inserts.

The function retrieves the subsystem information using the **IrInspectInformation** function. If the current condition equals to **IR_INFO_CONDITION_FAILURE,** then the function prints the following user-facing message and halts a machine using the **hlt** instruction: *"Vacui: Some kernel components failed to initialize!"*.

5.5. Inspection of the IR information

Visibility Scope: Public
Signature:

```
BOOL
IrInspectInformation(
        [out] IR_INFO* Information
);
```

Parameters:

- **Information –** A required pointer to a public information structure destination instance, which corresponds to the table *"3.1. Table - The IR Subsystem parameters"*. More fields will be added in the future as the table grows.

IR_INFO Structure:

- **IR_INFO_CONDITION Condition –** The current subsystem condition.

IR_INFO_CONDITION Enumeration:

- **IR_INFO_CONDITION_FAILURE**
- **IR_INFO_CONDITION_SUCCESS**
- **IR_INFO_CONDITION_STARTED**
- **IR_INFO_CONDITION_READY**

This function writes the IR subsystem information into the provided destination structure **Information**.

Returns:

- **TRUE** → The function successfully wrote the information.
- **FALSE** → The **Information** parameter is null.